

9

Image Segmentation

In the previous chapter, we learned about detecting objects present in images, along with the classes that correspond to the detected objects. In this chapter, we will go one step further by not only drawing a bounding box around an object but also by identifying the exact pixels that contain the object. In addition to that, by the end of this chapter, we will be able to single out instances/objects that belong to the same class.

We will also learn about semantic segmentation and instance segmentation by looking at the U-Net and Mask R-CNN architectures. Specifically, we will cover the following topics:

- Exploring the U-Net architecture
- Implementing semantic segmentation using U-Net to segment objects on a road
- Exploring the Mask R-CNN architecture
- Implementing instance segmentation using Mask R-CNN to identify multiple instances of a given class

A succinct illustration of what we are trying to achieve through image segmentation is as follows:

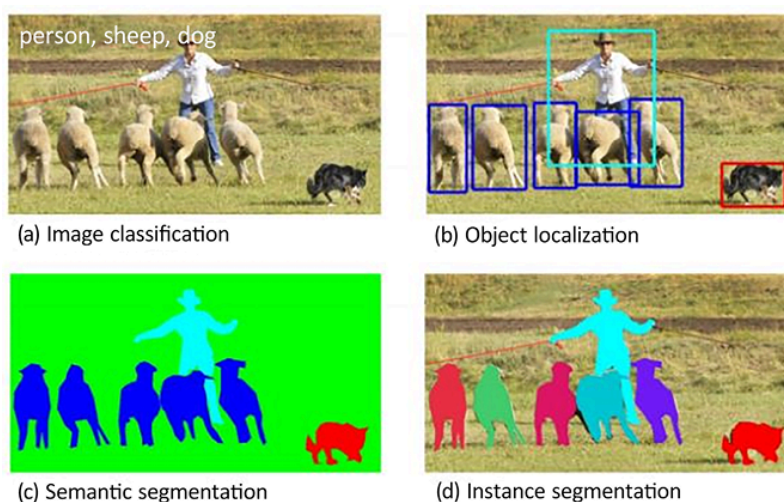


Figure 9.1: Difference between object localization & segmentation tasks on a sample image (source: <https://arxiv.org/pdf/1405.0312.pdf>)

The figure above shows: (a) object classes that are identified from an image, (b) object locations that are identified from an image, (c) object class masks that are identified from an image, and (d) object instances that are

segregated from an image. Now that you know what to expect, let's get started!



All the code in this chapter can be found in the **Chapter09** folder in the GitHub repository at <https://bit.ly/mcyp-2e>.

Exploring the U-Net architecture

Imagine a scenario where you're given an image and are asked to predict which pixel corresponds to what object. So far, when we have been predicting an object class and bounding box, we passed the image through a network, which then passes the image through a backbone architecture (such as VGG or ResNet), flattens the output at a certain layer, and connects additional dense layers before making predictions for the class and bounding-box offsets. However, in the case of image segmentation, where the output shape is the same as that of the input image's shape, flattening the convolutions' outputs and then reconstructing the image might result in a loss of information. Furthermore, the contours and shapes present in the original image will not vary in the output image in the case of image segmentation, so the networks we have dealt with so far (which flatten the last layer and connect additional dense layers) are not optimal when we perform segmentation.

In this section, we will learn how to perform image segmentation. The two aspects that we need to keep in mind while performing segmentation are as follows:

- The shape and structure of the objects in the original image remain the same in the segmented output.
- Leveraging a fully convolutional architecture (and not a structure where we flatten a certain layer) can help here, since we are using one image as input and another as output.

The U-Net architecture helps us achieve this. A typical representation of U-Net is as follows (the input image is of the shape 3 x 96 x 128, while the number of classes present in the image is 21; this means that the output contains 21 channels):

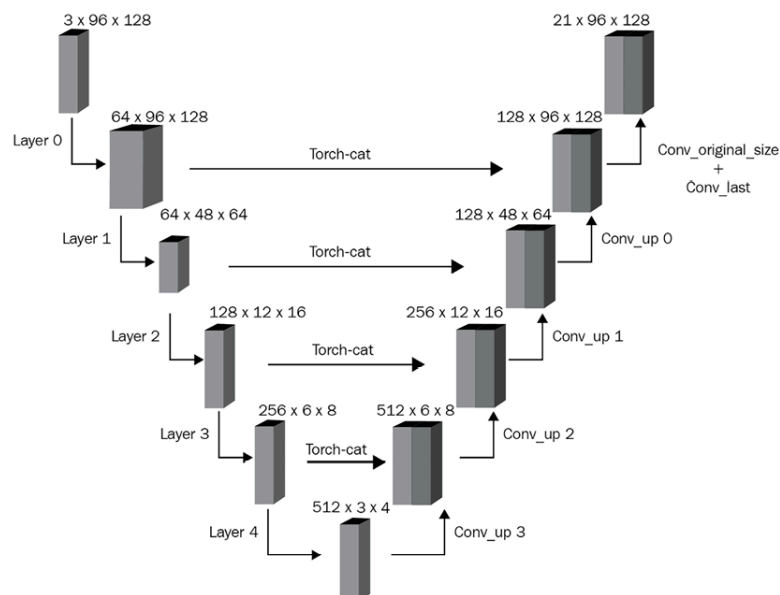


Figure 9.2: A sample U-Net architecture

The preceding architecture is called a **U-Net architecture** because of its U-like shape.

In the left half of the preceding diagram, we can see that the image passes through convolution layers, as we have seen in previous chapters, and that the image size keeps reducing while the number of channels keeps increasing. However, in the right half, we can see that we upscale the downscaled image, back to the original height and width but with as many channels as there are classes (21 in this case).

In addition, while upscaling, we also leverage information from the corresponding layers in the left half using **skip connections** (similar to the ones in ResNet that we learned about in *Chapter 5, Transfer Learning for Image Classification*) so that we can preserve the structure/objects in the original image. This way, the U-Net architecture learns to preserve the structure (and shapes of objects) of the original image while leveraging the convolution's features to predict the classes that correspond to each pixel.

In general, we have as many channels in the output as the number of classes we want to predict.

Now that we understand the high level of a U-Net architecture, let us understand the new concept we introduced, upscaling, in the next section.

Performing upscaling

In the U-Net architecture, upscaling is performed using the `nn.ConvTranspose2d` method, which takes the number of input channels, the number of output channels, the kernel size, and stride as input parameters. An example calculation for `ConvTranspose2d` is as follows:

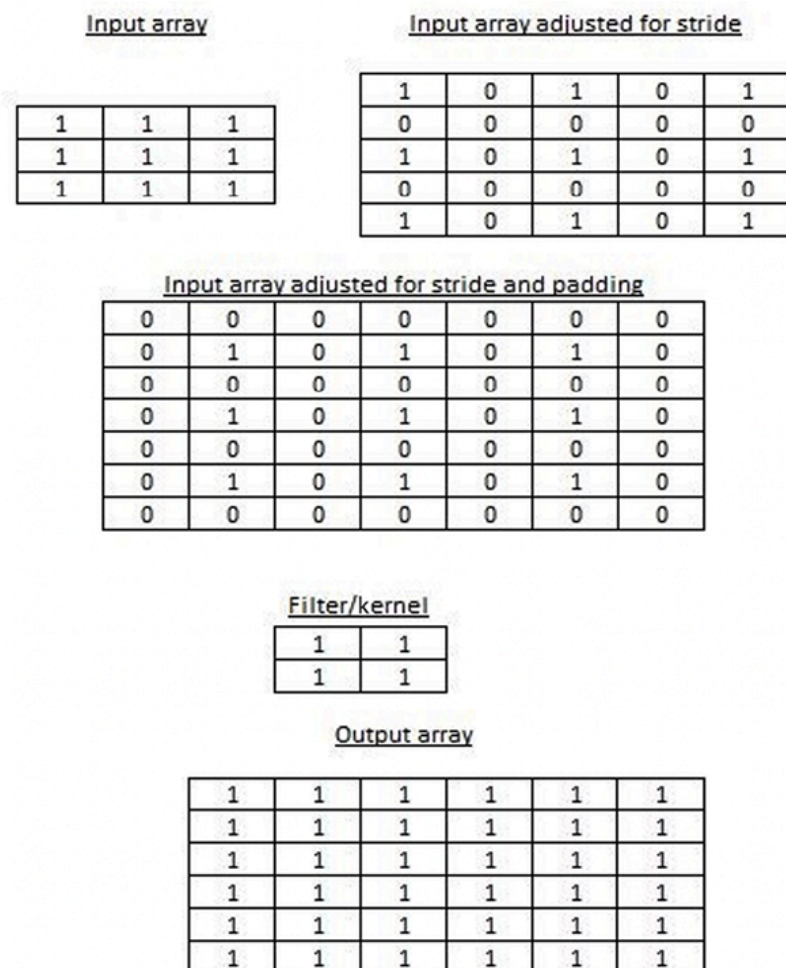


Figure 9.3: Upscaling operation

In the preceding example, we took an input array of shape 3 x 3 (**Input array**), applied a stride of 2 where we distributed the input values to accommodate the stride (**Input array adjusted for stride**), padded the array with zeros (**Input array adjusted for stride and padding**), and convolved the padded input with a filter (**Filter/Kernel**) to fetch the output array.



By leveraging a combination of padding and stride, we have upscaled an input that is 3 x 3 in shape to an array of 6 x 6 in shape. While the preceding example is only for illustration purposes, the optimal filter values learn (because the filter weights and bias are optimized during the model training process) to reconstruct the original image as much as possible.

The hyperparameters in `nn.ConvTranspose2d` are as follows:

```
help(nn.ConvTranspose2d)

Args:
  in_channels (int): Number of channels in the input image
  out_channels (int): Number of channels produced by the convolution
  kernel_size (int or tuple): Size of the convolving kernel
  stride (int or tuple, optional): Stride of the convolution. Default: 1
  padding (int or tuple, optional): ``dilation * (kernel_size - 1) - padding`` zero-padding
    will be added to both sides of each dimension in the input. Default: 0
  output_padding (int or tuple, optional): Additional size added to one side
    of each dimension in the output shape. Default: 0
  groups (int, optional): Number of blocked connections from input channels to output channels. Default: 1
  bias (bool, optional): If ``True``, adds a learnable bias to the output. Default: ``True``
  dilation (int or tuple, optional): Spacing between kernel elements. Default: 1
```

Figure 9.4: Arguments of `ConvTranspose2d`

In order to understand how `nn.ConvTranspose2d` helps upscale an array, let's go through the following code:

1. Import the relevant packages:

```
import torch
import torch.nn as nn
```

2. Initialize a network, `m`, with the `nn.ConvTranspose2d` method:

```
m = nn.ConvTranspose2d(1, 1, kernel_size=(2,2), stride=2, padding = 0)
```

In the preceding code, we are specifying that the input channel's value is `1`, the output channel's value is `1`, the size of the kernel is `(2,2)`, the stride is `2`, and the padding is `0`.

Internally, padding is calculated as $\text{dilation} * (\text{kernel_size} - 1) - \text{padding}$. Hence, it is $1 * (2 - 1) - 0 = 1$, where we add zero padding of 1 to both dimensions of the input array.

3. Initialize an input array and pass it through the model:

```
input = torch.ones(1, 1, 3, 3)
output = m(input)
output.shape
```

The preceding code results in a shape of `1x1x6x6`, as shown in the example image provided earlier.

Now that we understand how the U-Net architecture works and how `nn.ConvTranspose2d` helps upscale an image, let's implement it so that we can predict the different objects present in an image of a road scene.

Implementing semantic segmentation using U-Net

In this section, we'll leverage the U-Net architecture to predict the class that corresponds to all the pixels in the image. A sample of such an input-output combination is as follows:



Figure 9.5: (Left) input image; (Right) output image with classes corresponding to the various objects present in the image

Note that, in the preceding picture, the objects that belong to the same class (in the left image, the input image) have the same pixel value (in the right image, the output image), which is why we **segment** the pixels that are **semantically** similar to each other. This is also known as semantic segmentation. Let's learn how to code semantic segmentation:



Find the following code in the Semantic_Segmentation_with_U_Net.ipynb file in the Chapter09 folder on GitHub at <https://bit.ly/mcyp-2e>. The code contains URLs to download data from and is moderately lengthy.

1. Let's begin by downloading the necessary datasets, installing the necessary packages, and then importing them. Once we've done that, we can define the device:

```
import os
if not os.path.exists('dataset1'):
    !wget -q https://www.dropbox.com/s/0pigmmynbf9xwq/dataset1.zip
    !unzip -q dataset1.zip
    !rm dataset1.zip
    !pip install -q torch_snippets pytorch_model_summary
from torch_snippets import *
from torchvision import transforms
from sklearn.model_selection import train_test_split
device = 'cuda' if torch.cuda.is_available() else 'cpu'
```

2. Define the function that will be used to transform images (`tfms`):

```
tfms = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406],
                          [0.229, 0.224, 0.225])
])
```

3. Define the dataset class (`SegData`) to fetch input and output images for training:

1. Specify the folder that contains images in the `__init__` method:

```
class SegData(Dataset):
    def __init__(self, split):
        self.items=stems(f'dataset1/images_prepped_{split}')
        self.split = split
```

2. Define the `__len__` method:

```
def __len__(self):
    return len(self.items)
```

3. Define the `__getitem__` method:

```
def __getitem__(self, ix):
    image =read(f'dataset1/images_prepped_{self.split}/{self.items[ix]}.png', 1)
    image = cv2.resize(image, (224,224))
    mask=read(f'dataset1/annotations_prepped_{self.split}/{self.items[ix]}.png')[ :, :,0]
    mask = cv2.resize(mask, (224,224))
    return image, mask
```

In the `__getitem__` method, we are resizing both the input (`image`) and output (`mask`) images so that they're the same shape. Note that the mask images contain integers that fall in the range `[0,11]` . This indicates that there are 12 different classes.

4. Define a function (`choose`) to select a random image index (mainly for debugging purposes):

```
def choose(self): return self[randint(len(self))]
```

5. Define the `collate_fn` method to perform preprocessing on a batch of images:

```
def collate_fn(self, batch):
    ims, masks = list(zip(*batch))
    ims = torch.cat([tfms(im.copy()/255.)[None] \
                     for im in ims]).float().to(device)
    ce_masks = torch.cat([torch.Tensor(mask[None]) for \
                          mask in masks]).long().to(device)
    return ims, ce_masks
```

In the preceding code, we preprocess all the input images so that they have a channel (so that each image can be passed through a CNN later) once we've transformed the scaled images. Notice that `ce_masks` is a tensor of long integers, similar to the cross-entropy targets.

4. Define the training and validation datasets, as well as the dataloaders:


```
trn_ds = SegData('train')
val_ds = SegData('test')
trn_dl = DataLoader(trn_ds, batch_size=4, shuffle=True,
                    collate_fn=trn_ds.collate_fn)
val_dl = DataLoader(val_ds, batch_size=1, shuffle=True,
                    collate_fn=val_ds.collate_fn)
```

5. Define the neural network model architecture to train for segmentation. Note that a U-Net contains the traditional convolutions as well as up-convolution:

1. Define the convolution block (`conv`):

```
def conv(in_channels, out_channels):
    return nn.Sequential(
        nn.Conv2d(in_channels, out_channels, kernel_size=3,
                  stride=1, padding=1),
        nn.BatchNorm2d(out_channels),
        nn.ReLU(inplace=True)
    )
```

In the preceding definition of `conv`, we sequentially perform the `Conv2d` operation, the `BatchNorm2d` operation, and the `ReLU` operation.

2. Define the `up_conv` block:

```
def up_conv(in_channels, out_channels):
    return nn.Sequential(
        nn.ConvTranspose2d(in_channels, out_channels,
                           kernel_size=2, stride=2),
        nn.ReLU(inplace=True)
    )
```

`ConvTranspose2d` ensures that we upscale the images. This differs from the `Conv2d` operation, where we reduce the dimensions of the image. It takes an image that has `in_channels` number of channels as input channels and produces an image that has `out_channels` number of output channels.

3. Define the network class (`UNet`):

```
from torchvision.models import vgg16_bn
class UNet(nn.Module):
    def __init__(self, pretrained=True, out_channels=12):
        super().__init__()
        self.encoder = vgg16_bn(pretrained=pretrained).features
        self.block1 = nn.Sequential(*self.encoder[:6])
        self.block2 = nn.Sequential(*self.encoder[6:13])
        self.block3 = nn.Sequential(*self.encoder[13:20])
        self.block4 = nn.Sequential(*self.encoder[20:27])
        self.block5 = nn.Sequential(*self.encoder[27:34])
        self.bottleneck = nn.Sequential(*self.encoder[34:])
        self.conv_bottleneck = conv(512, 1024)
        self.up_conv6 = up_conv(1024, 512)
```



```

self.conv6 = conv(512 + 512, 512)
self.up_conv7 = up_conv(512, 256)
self.conv7 = conv(256 + 512, 256)
self.up_conv8 = up_conv(256, 128)
self.conv8 = conv(128 + 256, 128)
self.up_conv9 = up_conv(128, 64)
self.conv9 = conv(64 + 128, 64)
self.up_conv10 = up_conv(64, 32)
self.conv10 = conv(32 + 64, 32)
self.conv11 = nn.Conv2d(32, out_channels, kernel_size=1)

```

In the preceding `__init__` method, we define all the layers that we would use in the `forward` method.

4. Define the `forward` method:

```

def forward(self, x):
    block1 = self.block1(x)
    block2 = self.block2(block1)
    block3 = self.block3(block2)
    block4 = self.block4(block3)
    block5 = self.block5(block4)
    bottleneck = self.bottleneck(block5)
    x = self.conv_bottleneck(bottleneck)
    x = self.up_conv6(x)
    x = torch.cat([x, block5], dim=1)
    x = self.conv6(x)
    x = self.up_conv7(x)
    x = torch.cat([x, block4], dim=1)
    x = self.conv7(x)
    x = self.up_conv8(x)
    x = torch.cat([x, block3], dim=1)
    x = self.conv8(x)
    x = self.up_conv9(x)
    x = torch.cat([x, block2], dim=1)
    x = self.conv9(x)
    x = self.up_conv10(x)
    x = torch.cat([x, block1], dim=1)
    x = self.conv10(x)
    x = self.conv11(x)
    return x

```

In the preceding code, we make the U-style connection between the downscaling and upscaling convolution features by using `torch.cat` on the appropriate pairs of tensors.

5. Define a function (`UNetLoss`) that will calculate our loss and accuracy values:

```

ce = nn.CrossEntropyLoss()
def UnetLoss(preds, targets):
    ce_loss = ce(preds, targets)
    acc = (torch.max(preds, 1)[1] == targets).float().mean()
    return ce_loss, acc

```

6. Define a function that will train on a batch (`train_batch`) and calculate the metrics on the validation dataset (`validate_batch`):

```
def train_batch(model, data, optimizer, criterion):
    model.train()
    ims, ce_masks = data
    _masks = model(ims)
    optimizer.zero_grad()
    loss, acc = criterion(_masks, ce_masks)
    loss.backward()
    optimizer.step()
    return loss.item(), acc.item()

@torch.no_grad()
def validate_batch(model, data, criterion):
    model.eval()
    ims, masks = data
    _masks = model(ims)
    loss, acc = criterion(_masks, masks)
    return loss.item(), acc.item()
```

7. Define the model, optimizer, loss function, and the number of epochs for training:

```
model = UNet().to(device)
criterion = UnetLoss
optimizer = optim.Adam(model.parameters(), lr=1e-3)
n_epochs = 20
```

6. Train the model over increasing epochs:

```
log = Report(n_epochs)
for ex in range(n_epochs):
    N = len(trn_dl)
    for bx, data in enumerate(trn_dl):
        loss, acc = train_batch(model, data, optimizer, criterion)
        log.record(ex+(bx+1)/N, trn_loss=loss, trn_acc=acc, end='\r')
    N = len(val_dl)
    for bx, data in enumerate(val_dl):
        loss, acc = validate_batch(model, data, criterion)
        log.record(ex+(bx+1)/N, val_loss=loss, val_acc=acc, end='\r')

    log.report_avgs(ex+1)
```

7. Plot the training, validation loss, and accuracy values over increasing epochs:

```
log.plot_epochs(['trn_loss', 'val_loss'])
```

The preceding code generates the following output:

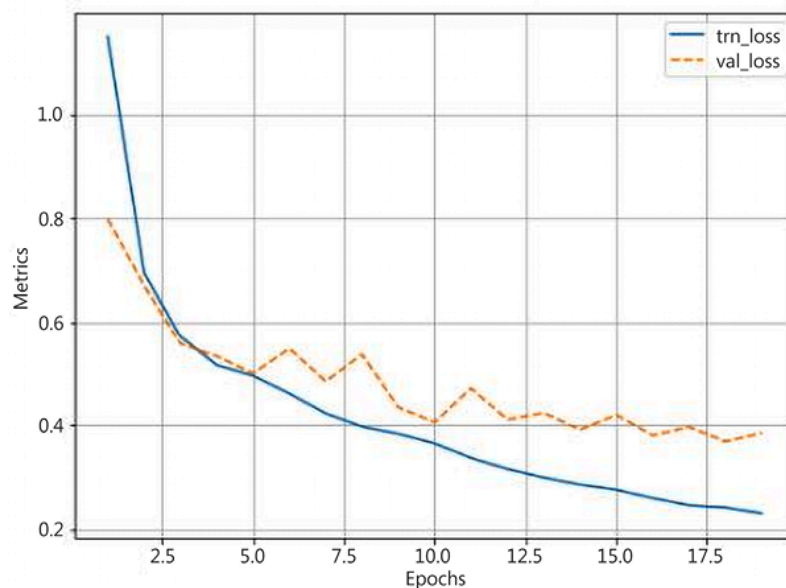


Figure 9.6: Training and validation loss over increasing epochs

8. Calculate the predicted output on a new image to observe model performance on unseen images:

1. Fetch model predictions on a new image:

```
im, mask = next(iter(val_dl))
_mask = model(im)
```

2. Fetch the channel that has the highest probability:

```
_, _mask = torch.max(_mask, dim=1)
```

3. Show the original and predicted images:

```
subplots([im[0].permute(1,2,0).detach().cpu()[ :, :, 0],
           mask.permute(1,2,0).detach().cpu()[ :, :, 0],
           _mask.permute(1,2,0).detach().cpu()[ :, :, 0]],nc=3,
          titles=['Original image','Original mask',
                  'Predicted mask'])
```

The preceding code generates the following output:



Figure 9.7: (Left) Original image; (Middle) original mask; (Right) predicted mask

From the preceding picture, we see that we can successfully generate a segmentation mask using the U-Net architecture. However, all instances of the same class will have the same predicted pixel value. What if we want to separate the instances of the `Person` class in the image?

In the next section, we will learn about the Mask R-CNN architecture, which helps to generate instance-level masks so that we can differentiate between instances (even instances of the same class).

Exploring the Mask R-CNN architecture

The Mask R-CNN architecture helps identify/highlight the instances of objects of a given class within an image. This becomes especially handy when there are multiple objects of the same type present within the image. Furthermore, the term **Mask** represents the segmentation that's done at the pixel level by Mask R-CNN.

The Mask R-CNN architecture is an extension of the Faster R-CNN network, which we learned about in the previous chapter. However, a few modifications have been made to the Mask R-CNN architecture, as follows:

- The `RoI Pooling` layer has been replaced with the `RoI Align` layer.
- A mask head has been included to predict a mask of objects in addition to the head, which already predicts the classes of objects and bounding-box correction in the final layer.
- A **fully convolutional network (FCN)** is leveraged for mask prediction.

Let's have a quick look at the events that occur within Mask R-CNN before we understand how each of the components works (image source:

<https://arxiv.org/pdf/1703.06870.pdf>):

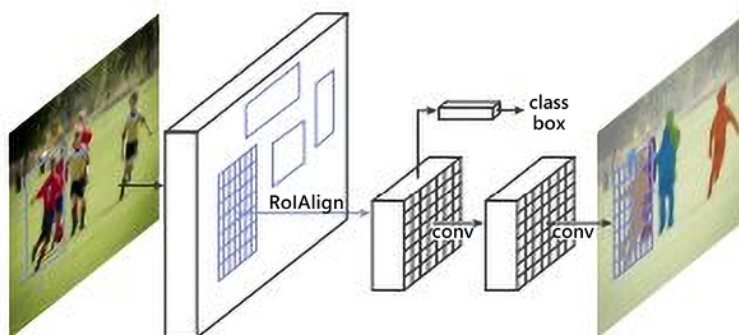


Figure 9.8: Mask R-CNN workflow

In the preceding diagram, note that we are fetching the class and bounding-box information from one layer and the mask information from another layer.

The working details of the Mask R-CNN architecture are as follows:

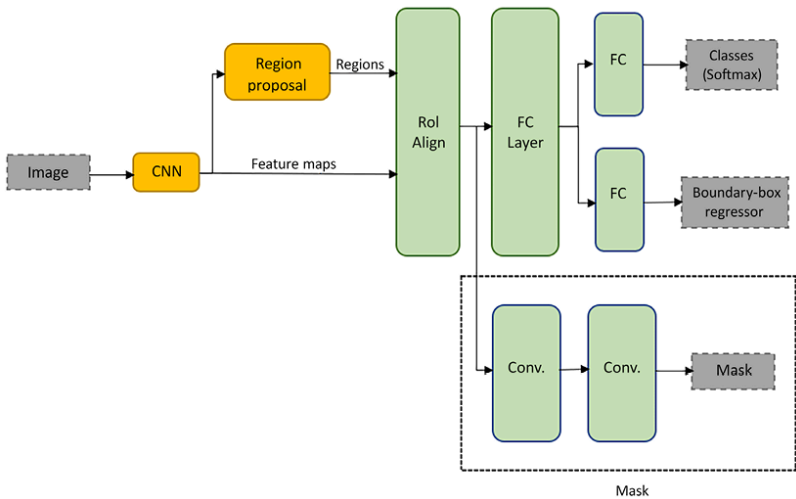


Figure 9.9: Working details of Mask R-CNN

Before we implement the Mask R-CNN architecture, we need to understand its components. We'll start with RoI Align.

RoI Align

With Faster R-CNN, we learned about RoI pooling. One of the drawbacks of RoI pooling is that we are likely to lose certain information when we perform the RoI pooling operation. This is because we are likely to have an uneven representation of content across all the areas of an image before pooling.

Let's go through the example we provided in the previous chapter:

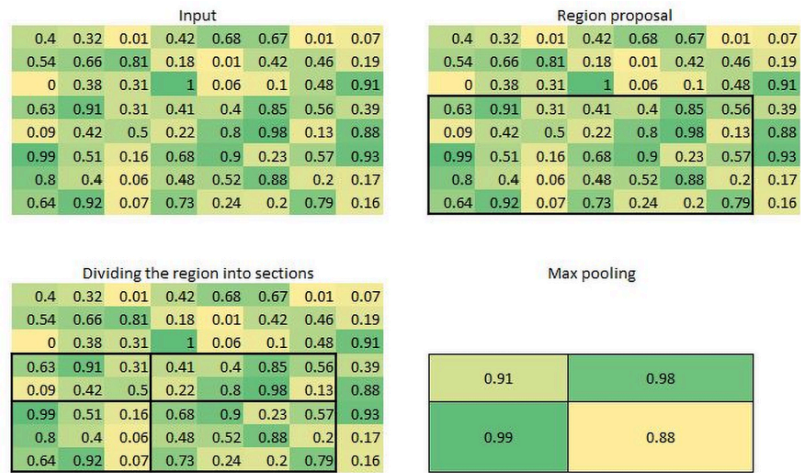


Figure 9.10: RoI pooling calculation

In the preceding image, the region proposal is 5 x 7 in shape, and we have to convert it into a 2 x 2 shape. While converting it into a 2 x 2 shape, one part of the region has less representation compared to other parts of the region. This results in information loss, since certain parts of the region have more weight than others. RoI Align comes to the rescue to address such a scenario.

Let's go through a simple example to understand how RoI Align works. Here, we try to convert the following region (which is represented by dashed lines) into a 2 x 2 shape:

0.4	0.08	0.73	0.57	0.13
0.88	0.13	0.32	0.64	0.15
0.98	0.66	0.16	0.16	0.25
0.97	0.45	0.08	0.08	0.18
0.69	0.88	0.9	0.9	0.87

Figure 9.11: Region represented with dashed lines

Note that the region (in dashed lines) is not equally spread across all the cells in the feature map.

We must perform the following steps to get a reasonable representation of the region in a 2 x 2 shape:

- 1. First, divide the region into an equal 2 x 2 shape:

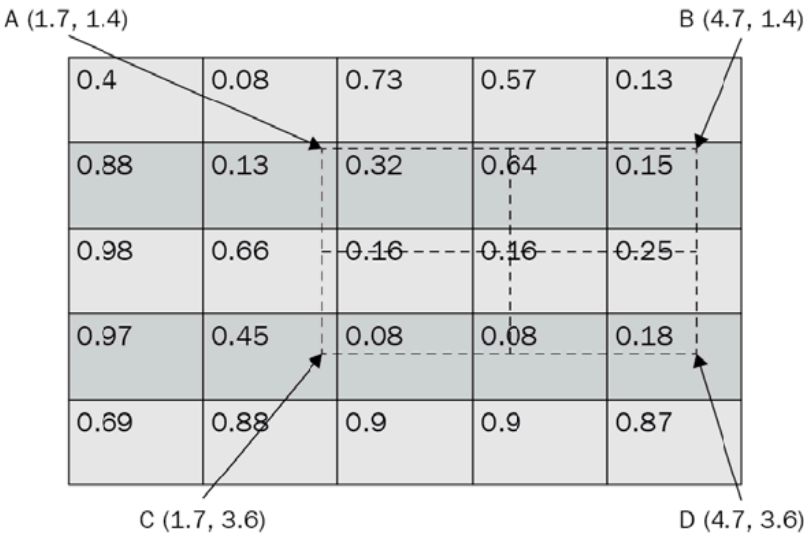


Figure 9.12: Calculating the four corners of the region

- Define four points that are equally spaced within each of the 2 x 2 cells:

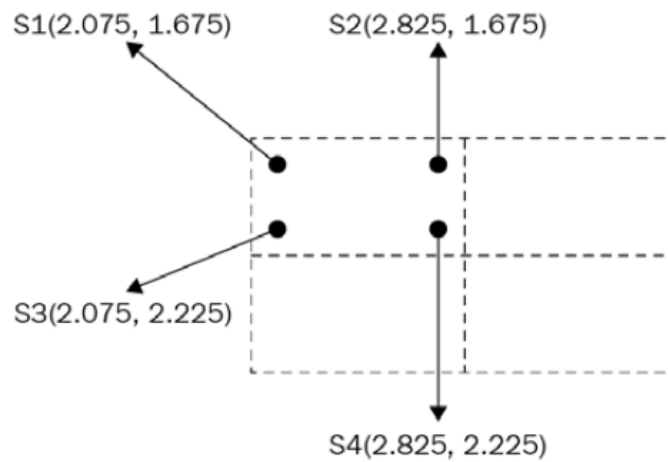


Figure 9.13: Calculating the four selected points

Note that, in the preceding diagram, the distance between two consecutive points is 0.75 (either horizontally or vertically).

- Calculate the weighted average value of each point based on its distance to the nearest known value:

0.4	0.08	0.73	0.57	0.13
0.88	0.13	P (0.32)	Q (0.64)	0.15
0.98	0.66	R (0.16)	S (0.12)	0.25
0.97	0.45	0.08	0.08	0.18
0.69	0.88	0.9	0.9	0.87

Figure 9.14: Extrapolating the value corresponding to each point

- Repeat the preceding interpolation step for all four points in a cell:

0.21778	0.27553	
0.14896	0.21852	

Figure 9.15: Values of the four corners

5. Perform average pooling across all four points within a cell:

0.2152	0.2335
0.3763	0.3562

Figure 9.16: Output of pooling for all the four cells

By implementing the preceding steps, we don't lose out on information when performing RoI Align, that is, when we place all the regions inside the same shape.

Mask head

Using RoI Align, we can get a more accurate representation of the region proposal that is obtained from the region proposal network. Now, we want to obtain the segmentation (mask) output, given a standard-shaped RoI Align output, for every region proposal.

Typically, in the case of object detection, we would pass the RoI Align through a flattened layer to predict the object's class and bounding-box offset. However, in the case of image segmentation, we predict the pixels within a bounding box that contains the object. Hence, we now have a third output (apart from the class and bounding-box offset), which is the predicted mask within the region of interest.

Here, we predict the mask, which is an image overlay on top of the original image. Given that we predict an image, instead of flattening the RoI Align's output, we'll connect it to another convolution layer to get another image-like structure (width x height in dimensions). Let's understand this phenomenon by looking at the following diagram:

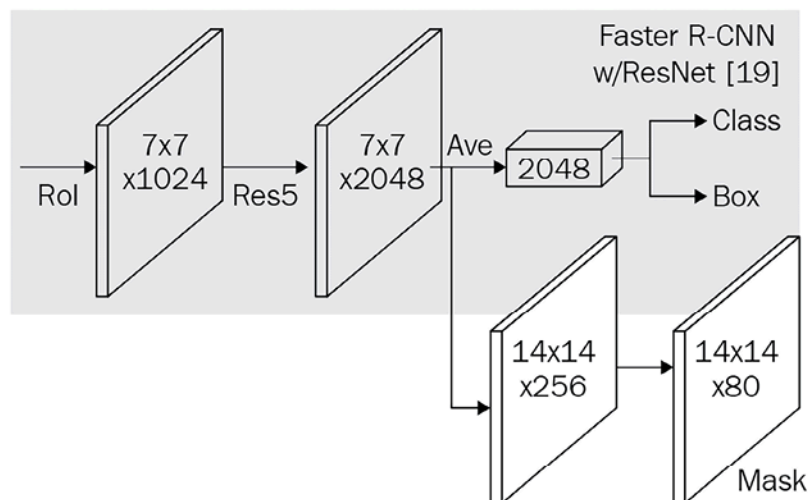


Figure 9.17: Workflow to obtain masks

In the preceding diagram, we have obtained an output of shape $7 \times 7 \times 2,048$ using the **feature pyramid network (FPN)**, which now has two branches:

- The first branch returns the class of the object and the bounding box, after flattening the FPN output.
- The second branch performs convolution on top of the FPN's output to get a mask.

The ground truth corresponding to the 14×14 output is the resized image of the region proposals. The ground truth of the region proposal is of the shape $80 \times 14 \times 14$ if there are 80 unique classes in the dataset. Each of the $80 \times 14 \times 14$ pixels is a 1 or a 0, which indicates whether the pixel contains an object or not. Thus, we are performing binary cross-entropy loss minimization while predicting the class of a pixel.

Post model training, we can detect regions, get classes, get bounding-box offsets, and get the mask corresponding to each region. When making an inference, we first detect the objects present in the image and make bounding-box corrections. Then, we pass the region offsets to the mask head to predict the mask that corresponds to different pixels in the region.

Now that we understand how the Mask R-CNN architecture works, let's code it up so that we can detect instances of people in an image.

Implementing instance segmentation using Mask R-CNN

To help us understand how to code Mask R-CNN for instance segmentation, we will leverage a dataset that masks people who are present within an image. The dataset we'll be using has been created from a subset of the ADE20K dataset that contains input images and their corresponding

masks, which is available at

<https://groups.csail.mit.edu/vision/datasets/ADE20K/>. We will only use those images where we have masks for people.

The strategy we'll adopt is as follows:

1. Fetch the dataset and then create datasets and dataloaders from it.
2. Create a ground truth in a format needed for PyTorch's official implementation of Mask R-CNN.
3. Download the pre-trained Faster R-CNN model and attach a Mask R-CNN head to it.
4. Train the model with a PyTorch code snippet that has been standardized to train Mask R-CNN.
5. Infer on an image by performing non-max suppression first and then identifying the bounding box and the mask corresponding to the people in the image.

Let's code up the preceding strategy:



Find the following code in the Instance_Segmentation.ipynb file in the Chapter09 folder on GitHub at <https://bit.ly/mcnp-2e>.

1. Import the relevant dataset and training utilities from GitHub:

```
!wget --quiet \
    http://sceneparsing.csail.mit.edu/data/ChallengeData2017/images.tar
!wget --quiet http://sceneparsing.csail.mit.edu/data/ChallengeData2017/annotations_instanc
!tar -xf images.tar
!tar -xf annotations_instance.tar
!rm images.tar annotations_instance.tar
!pip install -qU torch_snippets
!wget --quiet https://raw.githubusercontent.com/pytorch/vision/master/references/detection
!wget --quiet https://raw.githubusercontent.com/pytorch/vision/master/references/detection
!wget --quiet https://raw.githubusercontent.com/pytorch/vision/master/references/detection
!wget --quiet https://raw.githubusercontent.com/pytorch/vision/master/references/detection
!wget --quiet https://raw.githubusercontent.com/pytorch/vision/master/references/detection
!pip install -q -U \
    'git+https://github.com/cocodataset/cocoapi.git#subdirectory=PythonAPI'
```

2. Import all the necessary packages and define device:

```
from torch_snippets import *
import torchvision
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
from torchvision.models.detection.mask_rcnn import MaskRCNNPredictor
from engine import train_one_epoch, evaluate
import utils
import transforms as T
device = 'cuda' if torch.cuda.is_available() else 'cpu'
```

3. Fetch images that contain masks of people, as follows:

1. Loop through the `images` and `annotations_instance` folders to fetch filenames:

```
all_images = Glob('images/training')
all_annots = Glob('annotations_instance/training')
```

2. Inspect the original image and the representation of masks of instances of people:

```
f = 'ADE_train_00014301'
im = read(find(f, all_images), 1)
an = read(find(f, all_annots), 1).transpose(2,0,1)
r,g,b = an
nzs = np.nonzero(r==4) # 4 stands for person
instances = np.unique(g[nzs])
masks = np.zeros((len(instances), *r.shape))
for ix,_id in enumerate(instances):
    masks[ix] = g==_id
subplots([im, *masks], sz=20)
```

The preceding code generates the following output. We can see that a separate mask has been generated for each person. Here, there are four instances of the `Person` class:



Figure 9.18: Separate mask generation for each individual

In this particular dataset, the ground-truth instance annotations are provided in such a way that the Red channel in RGB corresponds to the class of object, while the Green channel corresponds to the instance number (if there are multiple objects of the same class in the image, as in our example here). Furthermore, the `Person` class is encoded with a value of 4.

3. Loop through the annotations and store the files that contain at least one person:

```
annots = []
for ann in Tqdm(all_annots):
    _ann = read(ann, 1).transpose(2,0,1)
    r,g,b = _ann
    if 4 not in np.unique(r): continue
    annots.append(ann)
```

4. Split the files into training and validation files:

```
from sklearn.model_selection import train_test_split
_annots = stems(annots)
trn_items, val_items = train_test_split(_annots, random_state=2)
```

4. Define the transformation method:

```
def get_transform(train):
    transforms = []
    transforms.append(T.PILToTensor())
    if train:
        transforms.append(T.RandomHorizontalFlip(0.5))
    return T.Compose(transforms)
```

5. Create the dataset class (`MasksDataset`), as follows:

1. Define the `__init__` method, which takes the image names (`items`), transformation method (`transforms`), and the number of files to consider (`N`) as input:

```
class MasksDataset(Dataset):
    def __init__(self, items, transforms, N):
        self.items = items
        self.transforms = transforms
        self.N = N
```

2. Define a method (`get_mask`) that will fetch the number of masks that's equivalent to the instances present in the image:

```
def get_mask(self, path):
    an = read(path, 1).transpose(2,0,1)
    r,g,b = an
    nzs = np.nonzero(r==4)
    instances = np.unique(g[nzs])
    masks = np.zeros((len(instances), *r.shape))
    for ix,_id in enumerate(instances):
        masks[ix] = g==_id
    return masks
```

3. Fetch the image and the corresponding target values to be returned. Each person (instance) is treated as a different object class; that is, each instance is a different class. Note that, similar to training the Faster R-CNN model, the targets are returned as a dictionary of tensors. Let's define the `__getitem__` method:

```
def __getitem__(self, ix):
    _id = self.items[ix]
    img_path = f'images/training/{_id}.jpg'
    mask_path = f'annotations_instance/training/{_id}.png'
    masks = self.get_mask(mask_path)
    obj_ids = np.arange(1, len(masks)+1)
```

```
img = Image.open(img_path).convert("RGB")
num_objs = len(obj_ids)
```

4. Apart from the masks themselves, Mask R-CNN also needs the bounding-box information. However, this is easy to prepare, as shown in the following code:

```
boxes = []
for i in range(num_objs):
    obj_pixels = np.where(masks[i])
    xmin = np.min(obj_pixels[1])
    xmax = np.max(obj_pixels[1])
    ymin = np.min(obj_pixels[0])
    ymax = np.max(obj_pixels[0])
    if ((xmax-xmin)<=10) | (ymax-ymin)<=10):
        xmax = xmin+10
        ymax = ymin+10
    boxes.append([xmin, ymin, xmax, ymax])
```

In the preceding code, we adjust for scenarios where there are dubious ground truths (i.e., the height or width of the `Person` class is less than 10 pixels) by adding 10 pixels to the minimums of the `x` and `y` coordinates of the bounding box.

5. Convert all the target values into tensor objects:

```
boxes = torch.as_tensor(boxes, dtype=torch.float32)
labels = torch.ones((num_objs,), dtype=torch.int64)
masks = torch.as_tensor(masks, dtype=torch.uint8)
area = (boxes[:, 3] - boxes[:, 1]) * (boxes[:, 2] - boxes[:, 0])
iscrowd = torch.zeros((num_objs,), dtype=torch.int64)
image_id = torch.tensor([ix])
```

6. Store the target values in a dictionary:

```
target = {}
target["boxes"] = boxes
target["labels"] = labels
target["masks"] = masks
target["image_id"] = image_id
target["area"] = area
target["iscrowd"] = iscrowd
```

7. Specify the transformation method and return the image after scaling it:

```
if self.transforms is not None:
    img, target = self.transforms(img, target)
if (img.dtype == torch.float32) or (img.dtype == torch.uint8):
    img = img/255.
return img, target
```

8. Specify the `__len__` method:

```
def __len__(self):
    return self.N
```

9. Define the function that will choose a random image:

```
def choose(self):
    return self[randint(len(self))]
```

10. Inspect an input-output combination:

```
x = MasksDataset(trn_items, get_transform(train=True), N=100)
im, targ = x[0]
inspect(im, targ)
subplots([im, *targ['masks']], sz=10)
```

The following is some example output that the preceding code produces when it's run. We can see that the mask's shape is 2 x 512 x 683, indicating that there are two people in the image:

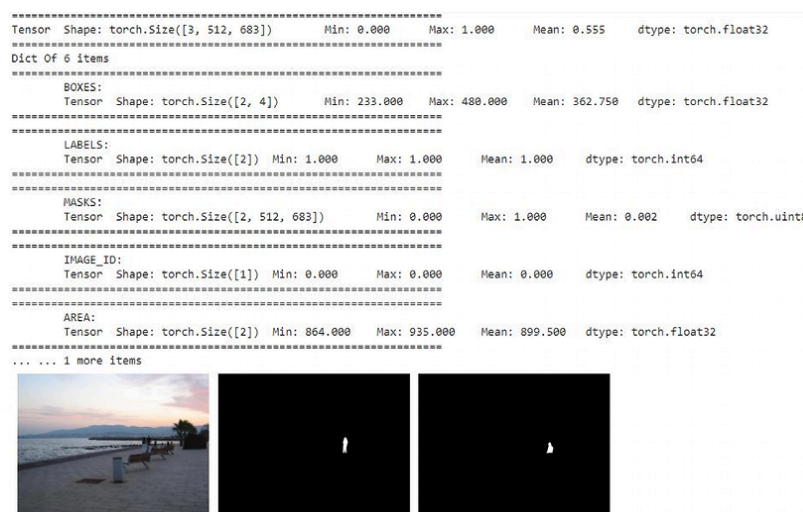


Figure 9.19: (Left) Input image; (Middle and right), predicted masks of persons

Note that, in the `__getitem__` method, we have as many masks and bounding boxes in an image as there are objects (instances) present within the image. Furthermore, because we only have two classes (the `Background` class and the `Person` class), we specify the `Person` class as 1.

By the end of this step, we have quite a lot of information in the output dictionary, that is, the object classes, bounding boxes, masks, the area of the masks, and if a mask corresponds to a crowd. All of this information is available in the `target` dictionary. For the training function that we

are going to use, it is important for the data to be standardized in the format that the `torchvision.models.detection.maskrcnn_resnet50_fpn` class requires it to be in.

6. Next, we need to define the instance segmentation model (`get_model_instance_segmentation`). We are going to use a pre-trained model with only the heads reinitialized to predict two classes (background and person). First, we need to initialize a pre-trained model and replace the `box_predictor` and `mask_predictor` heads so that they can be learned from scratch:

```
def get_model_instance_segmentation(num_classes):
    # Load an instance segmentation model pre-trained on
    # COCO
    model = torchvision.models.detection\
        .maskrcnn_resnet50_fpn(pretrained=True)

    # get number of input features for the classifier
    in_features = model.roi_heads.box_predictor.cls_score.in_features
    # replace the pre-trained head with a new one
    model.roi_heads.box_predictor = FastRCNNPredictor(\
        in_features, num_classes)

    in_features_mask = model.roi_heads\
        .mask_predictor.conv5_mask.in_channels
    hidden_layer = 256
    # and replace the mask predictor with a new one
    model.roi_heads.mask_predictor = MaskRCNNPredictor(\
        in_features_mask,
        hidden_layer,
        num_classes)

    return model
```

`FastRCNNPredictor` expects two inputs: `in_features` (the number of input channels) and `num_classes` (the number of classes). Based on the number of classes to predict, the number of bounding box predictions is calculated, which is four times the number of classes.

`MaskRCNNPredictor` expects three inputs: `in_features_mask` (the number of input channels), `hidden_layer` (the number of channels in the output), and `num_classes` (the number of classes to predict).

Details of the defined model can be obtained by specifying the following:

```
model = get_model_instance_segmentation(2).to(device)
model
```

The bottom half of the model (that is, without the backbone) would look like this:

```

(roi_heads): RoIHeads(
  (box_roi_pool): MultiScaleRoIAlign()
  (box_head): TwoMLPHead(
    (fc6): Linear(in_features=12544, out_features=1024, bias=True)
    (fc7): Linear(in_features=1024, out_features=1024, bias=True)
  )
  (box_predictor): FastRCNNPredictor(
    (cls_score): Linear(in_features=1024, out_features=2, bias=True)
    (bbox_pred): Linear(in_features=1024, out_features=8, bias=True)
  )
  (mask_roi_pool): MultiScaleRoIAlign()
  (mask_head): MaskRCNNHeads(
    (mask_fcn1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (relu1): ReLU(inplace=True)
    (mask_fcn2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (relu2): ReLU(inplace=True)
    (mask_fcn3): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (relu3): ReLU(inplace=True)
    (mask_fcn4): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (relu4): ReLU(inplace=True)
  )
  (mask_predictor): MaskRCNNPredictor(
    (conv5_mask): ConvTranspose2d(256, 256, kernel_size=(2, 2), stride=(2, 2))
    (relu): ReLU(inplace=True)
    (mask_fcn_logits): Conv2d(256, 2, kernel_size=(1, 1), stride=(1, 1))
  )
)

```

Figure 9.20: Mask R-CNN model architecture

Note that the major difference between the Faster R-CNN (which we trained in the previous chapter) and the Mask R-CNN model is in the `roi_heads` module, which itself contains multiple sub-modules. Let's take a look at what tasks they perform:

- `roi_heads` : Aligns the inputs taken from the FPN network and creates two tensors.
- `box_predictor` : Uses the outputs we obtained to predict classes and bounding-box offsets for each RoI.
- `mask_roi_pool` : This layer then aligns the outputs coming from the FPN.
- `mask_head` : Converts the aligned outputs obtained previously into feature maps that can be used to predict masks.
- `mask_predictor` : Takes the outputs from `mask_head` and predicts the final masks.

7. Fetch the dataset and dataloaders that correspond to the train and validation images:

```

dataset = MasksDataset(trn_items, get_transform(train=True), N=3000)
dataset_test = MasksDataset(val_items, get_transform(train=False), N=800)
# define training and validation data loaders
data_loader=torch.utils.data.DataLoader(dataset,
                                         batch_size=2, shuffle=True, num_workers=0,
                                         collate_fn=utils.collate_fn)
data_loader_test = torch.utils.data.DataLoader(dataset_test,
                                                batch_size=1, shuffle=False,
                                                num_workers=0, collate_fn=utils.collate_fn)

```

8. Define the model, parameters, and optimization criterion:

```

num_classes = 2
model = get_model_instance_segmentation(num_classes).to(device)
params = [p for p in model.parameters() if p.requires_grad]
optimizer = torch.optim.SGD(params, lr=0.005,
                             momentum=0.9, weight_decay=0.0005)
# and a learning rate scheduler
lr_scheduler = torch.optim.lr_scheduler.StepLR(optimizer,
                                                step_size=3, gamma=0.1)

```

The defined pre-trained model architecture takes the image and the `targets` dictionary as input to reduce loss. A sample of the output that will be received from the model can be seen by running the following command:

```

# The following code is for illustration purpose only
model.eval()
pred = model(dataset[0][0][None].to(device))
inspect(pred[0])

```

The preceding code results in the following output:

```

=====
Dict Of 4 items
=====
BOXES:
Tensor Shape: torch.Size([100, 4]) Min: 0.000 Max: 1024.000 Mean: 385.767 dtype: torch.float32
=====
LABELS:
Tensor Shape: torch.Size([100]) Min: 1.000 Max: 1.000 Mean: 1.000 dtype: torch.int64
=====
SCORES:
Tensor Shape: torch.Size([100]) Min: 0.491 Max: 0.648 Mean: 0.531 dtype: torch.float32
=====
MASKS:
Tensor Shape: torch.Size([100, 1, 692, 1024]) Min: 0.000 Max: 1.000 Mean: 0.012 dtype: torch.float32
=====

```

Figure 9.21: Sample predictions

Here, we can see a dictionary with bounding boxes (`BOXES`), classes corresponding to bounding boxes (`LABELS`), confidence scores corresponding to class predictions (`SCORES`), and the location of our mask instances (`MASKS`). As you can see, the model is hardcoded to return 100 predictions, which is reasonable, since we shouldn't expect more than 100 objects in a typical image.

To fetch the number of instances that have been detected, we would use the following code:

```

# The following code is for illustration purpose only
pred[0]['masks'].shape
# torch.Size([100, 1, 536, 559])

```

The preceding code fetches a maximum of 100 mask instances (where the instances correspond to a non-background class) for an image (along with the dimensions corresponding to the image). For these 100 instances, it

would also return the corresponding class label, the bounding box, and the 100 corresponding confidence values of the class.

9. Train the model over increasing epochs:

```
num_epochs = 5
trn_history = []
for epoch in range(num_epochs):
    # train for one epoch, printing every 10 iterations
    res = train_one_epoch(model, optimizer, data_loader,
                          device, epoch, print_freq=10)
    trn_history.append(res)
    # update the learning rate
    lr_scheduler.step()
    # evaluate on the test dataset
    res = evaluate(model, data_loader_test, device=device)
```

By doing this, we can now overlay our masks over people in an image. We can log our training loss variation over increasing epochs as follows:

```
import matplotlib.pyplot as plt
plt.title('Training Loss')
losses=[np.mean(list(trn_history[i].meters['loss'].deque)) \
        for i in range(len(trn_history))]
plt.plot(losses)
```

The preceding code results in the following output:



Figure 9.22: Training loss over increasing epochs

10. Predict on a test image:

```
model.eval()
im = dataset_test[0][0]
```

```

show(im)
with torch.no_grad():
    prediction = model([im.to(device)])
    for i in range(len(prediction[0]['masks'])):
        plt.imshow(Image.fromarray(prediction[0]['masks']\
                                   [i, 0].mul(255).byte().cpu().numpy()))
        plt.title('Class: '+str(prediction[0]['labels']\
                                [i].cpu().numpy())+' Score:'+str(\
                                prediction[0]['scores'][i].cpu().numpy()))
    plt.show()

```

The preceding code results in the following output. We can see that we can successfully identify the four people in the image. Furthermore, the model predicts multiple other segments in the image (which we have not shown in the preceding output), although this is with low confidence:

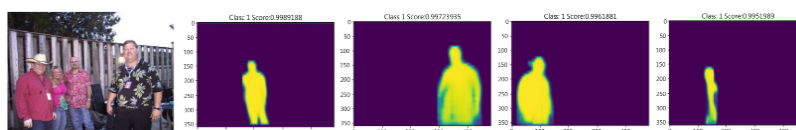


Figure 9.23: (Left) Input image; (rest) predicted masks

Now that the model can detect instances well, let's run predictions on a custom image that is not present within the provided dataset.

11. Run predictions on a new image of your own:

```

!wget https://www.dropbox.com/s/e92sui3a4kt/Hema18.JPG
img = Image.open('Hema18.JPG').convert("RGB")
from torchvision import transforms
pil_to_tensor = transforms.ToTensor()(img).unsqueeze_(0)
Image.fromarray(pil_to_tensor[0].mul(255)\
                .permute(1, 2, 0).byte().numpy())

```

The input image is as follows:



Figure 9.24: Sample input image outside of validation data

12. Fetch predictions on the input image:

```
model.eval()
with torch.no_grad():
    prediction = model([pil_to_tensor[0].to(device)])
    for i in range(len(prediction[0]['masks'])):
        plt.imshow(Image.fromarray(prediction[0]['masks'][i, 0].mul(255).byte().cpu().numpy()))
        plt.title('Class: '+str(prediction[0]\
            ['labels'][i].cpu().numpy())+ \
            'Score: '+str(prediction[0]['scores'][i].cpu().numpy()))
        plt.show()
```

The preceding code results in the following output:

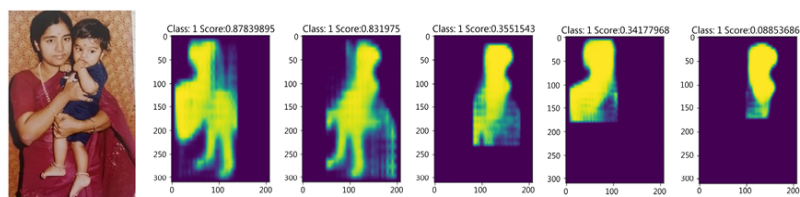


Figure 9.25: (Left) Input image; (rest) predicted masks

Note that, in the preceding image, the trained model did not work as well as it did on the test images. This could be due to the following reasons:

- The people might not have been in such close proximity and overlapped during training.
- The model might not have been trained on as many images where the classes of interest occupy the majority of the image.

- The images in the dataset that we have trained our model on have a different data distribution from the image being predicted on.

However, even though duplicate masks have been detected, having lower class scores in those regions (starting with the third mask) is a good indicator that there might be duplicates in predictions.

So far, we have learned about segmenting multiple instances of the `Person` class. In the next section, we will learn about what we need to tweak in the code we built in this section to segment multiple instances of multiple classes of objects in an image.

Predicting multiple instances of multiple classes

In the previous section, we learned about segmenting the `Person` class. In this section, we will learn about segmenting for person and table instances in one go by using the same model we built in the previous section. Let's get started:



Given that the majority of the code remains the same as it was in the previous section, we will only explain the additional code within this section. While executing code, we encourage you to go through the `predicting_multiple_instances_of_multiple_classes.ipynb` notebook, which can be found in the `Chapter09` folder on GitHub at <https://bit.ly/mcyp-2e>.

1. Fetch images that contain the classes of interest – `Person` (class ID 4) and `Table` (class ID 6):

```
classes_list = [4,6]
annots = []
for ann in Tqdm(all_annots):
    _ann = read(ann, 1).transpose(2,0,1)
    r,g,b = _ann
    if np.array([num in np.unique(r) for num in \
                classes_list]).sum()==0: continue
    annots.append(ann)
from sklearn.model_selection import train_test_split
_annots = stems(annots)
trn_items, val_items = train_test_split(_annots, random_state=2)
```

In the preceding code, we fetch the images that contain at least one of the classes of interest (`classes_list`).

2. Modify the `get_mask` method so that it returns both masks, as well as the classes that correspond to each mask in the `MasksDataset` class:

```
def get_mask(self, path):
    an = read(path, 1).transpose(2, 0, 1)
    r, g, b = an
    cls = list(set(np.unique(r)).intersection({4, 6}))
    masks = []
    labels = []
    for _cls in cls:
        nzs = np.nonzero(r==_cls)
        instances = np.unique(g[nzs])
        for ix, _id in enumerate(instances):
            masks.append(g[_id])
            labels.append(classes_list.index(_cls)+1)
    return np.array(masks), np.array(labels)
```

In the preceding code, we fetch the classes of interest that exist within the image and store them in `cls`. Next, we loop through each identified class (`cls`) and store the locations where the red channel values correspond to the class (`cls`) in `nzs`. Then, we fetch the instance IDs (`instances`) in those locations. Furthermore, we append `instances` to `masks` and the classes corresponding to instances in `labels` before returning the NumPy arrays for `masks` and `labels`.

3. Modify the `labels` object in the `__getitem__` method so that it contains labels that have been obtained from the `get_mask` method, instead of filling it with `torch.ones`. The bold part of the following code is where this change was implemented on the `__getitem__` method in the previous section:

```
def __getitem__(self, ix):
    _id = self.items[ix]
    img_path = f'images/training/{_id}.jpg'
    mask_path = f'annotations_instance/training/{_id}.png'
    masks, labels = self.get_mask(mask_path)
    #print(labels)
    obj_ids = np.arange(1, len(masks)+1)
    img = Image.open(img_path).convert("RGB")
    num_objs = len(obj_ids)
    boxes = []
    for i in range(num_objs):
        obj_pixels = np.where(masks[i])
        xmin = np.min(obj_pixels[1])
        xmax = np.max(obj_pixels[1])
        ymin = np.min(obj_pixels[0])
        ymax = np.max(obj_pixels[0])
        if ((xmax-xmin)<=10) | (ymax-ymin)<=10):
            xmax = xmin+10
            ymax = ymin+10
        boxes.append([xmin, ymin, xmax, ymax])
    boxes = torch.as_tensor(boxes, dtype=torch.float32)
    labels = torch.as_tensor(labels, dtype=torch.int64)
    masks = torch.as_tensor(masks, dtype=torch.uint8)
    area = (boxes[:, 3] - boxes[:, 1]) * (boxes[:, 2] - boxes[:, 0])
    iscrowd = torch.zeros((num_objs,), dtype=torch.int64)
```



```

image_id = torch.tensor([ix])
target = {}
target["boxes"] = boxes
target["labels"] = labels
target["masks"] = masks
target["image_id"] = image_id
target["area"] = area
target["iscrowd"] = iscrowd
if self.transforms is not None:
    img, target = self.transforms(img, target)
    if (img.dtype == torch.float32) or (img.dtype == torch.uint8) :
        img = img/255.
return img, target
def __len__(self):
    return self.N
def choose(self):
    return self[randint(len(self))]

```

4. Specify that you have three classes instead of two while defining `model`, as we now have person, table, and background classes to predict:

```

num_classes = 3
model=get_model_instance_segmentation(num_classes).to(device)

```

Upon training the model, as we did in the previous section, we'll see that the variation of training loss over increasing epochs is as follows:

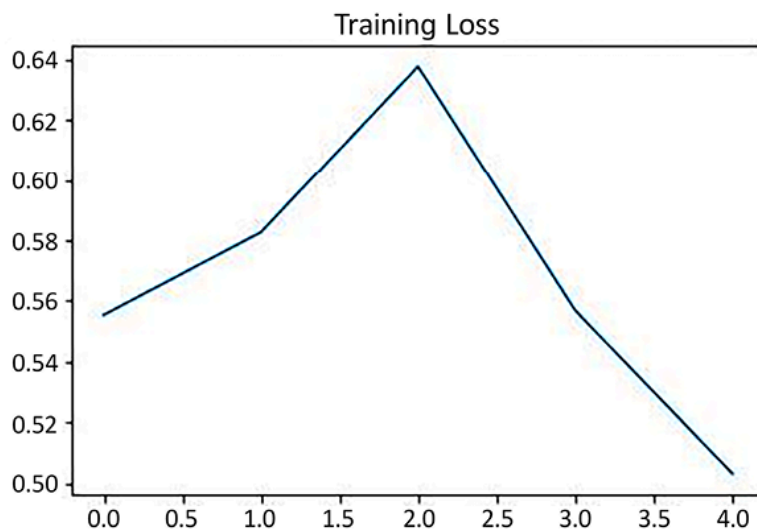


Figure 9.26: Training loss over increasing epochs

Furthermore, the predicted segments for a sample image that contains a person and a table are as follows:

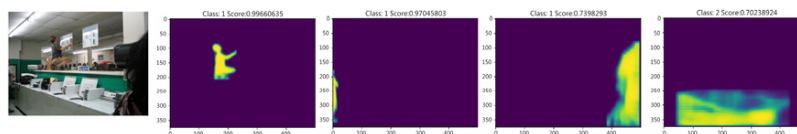


Figure 9.27: (Left) Input image; (rest) predicted masks and the corresponding class

From the preceding figure, we can see that we are able to predict both classes using the same model. As an exercise, we encourage you to increase the number of classes and the number of epochs and see what results you get.

Summary

In this chapter, we learned how to leverage U-Net and Mask R-CNN to perform segmentation on top of images. We understood how the U-Net architecture can perform downscaling and upscaling on images using convolutions to retain the structure of an image, while still being able to predict masks around objects within the image. We then cemented our understanding of this using the road scene detection exercise, where we segmented the image into multiple classes. Then, we learned about RoI Align, which helps ensure that the issues with RoI pooling surrounding image quantization are addressed. After that, we learned how Mask R-CNN works so that we could train models to predict instances of people in images, as well as instances of people and tables in an image.

Now that we have a good understanding of various object detection techniques and image segmentation techniques, in the next chapter, we will learn about applications that leverage the techniques we have learned about so far so that we can expand the number of classes that we will predict. In addition, we will also learn about the Detectron2 framework, which reduces code complexity while we build Faster R-CNN and Mask R-CNN models.

Questions

1. How does upscaling help in the U-Net architecture?
2. Why do we need to have a fully convolutional network in U-Net?
3. How does RoI Align improve upon RoI pooling in Mask R-CNN?
4. What is the major difference between U-Net and Mask R-CNN for segmentation?
5. What is instance segmentation?

Learn more on Discord

Join our community's Discord space for discussions with the authors and other readers:

<https://packt.link/modcv>

